

Data Compression Course Project

Implementation of the BZip2 Compression Algorithm

Project Report

Course Instructor: Dr. Faisal Aslam

Submission Year: 2026

Encoding pipeline:

Input → Block Division → RLE-1 → BWT → MTF → RLE-2 → Huffman → Compressed Output

Repository: [project-bzip2/](#)

Source language: C (C11) · Build: Makefile (Linux + Windows / MSYS2)

Benchmarks: Calgary + Canterbury + Silesia + large text/binary

1. Introduction & Repository Layout

1. Project Overview

We implemented a simplified BZip2 compression pipeline in C, satisfying every stage and prototype required by the course description: block division, RLE-1, Burrows-Wheeler Transform (matrix and suffix-array), Move-to-Front, RLE-2 and canonical Huffman coding. Both encoding and decoding paths are implemented, and every block is verified by an end-to-end round-trip (encode -> decode -> memcmp with the original).

1.1 Pipeline

Input -> Block Division -> RLE-1 -> BWT -> MTF -> RLE-2 -> Huffman

1.2 Repository layout (matches submission guideline section 9.1):

src/ - main.c, pipeline.c, block.c, rle1.c, bwt.c, mtf.c,
rle2.c, huffman.c, range.c, config.c
include/ - one header per module (block.h, bwt.h, ..., bzip2.h)
benchmarks/ - Calgary + Canterbury + Silesia + large text/binary
results/ - results.csv, compression_ratio.png
Makefile, config.ini, README.md, benchmark.py, project.pdf

1.3 Build and run

Linux/macOS: `make && ./bzip2_impl && python3 benchmark.py --no-run`

Windows MSYS2: `mingw32-make && ./bzip2_impl.exe && python benchmark.py --no-run`

Cross compile: `make windows CC=x86_64-w64-mingw32-gcc`

2. Configuration

2. Configuration Management (section 2.2)

Settings are read from config.ini at start-up. Defaults are applied if the file is missing.

[General]

```
block_size      = 900000      ; 100 KB - 900 KB
rle1_enabled    = true
bwt_type        = suffix_array ; matrix | suffix_array
mtf_enabled     = true
rle2_enabled    = true
huffman_enabled = true
```

[Performance]

```
benchmark_mode = false
output_metrics = true
```

[Paths]

```
input_directory = ./benchmarks/
output_directory = ./results/
```

Implementation note: every stage can be toggled independently which lets us regression-test sub-pipelines. We default to suffix-array BWT because the matrix-rotation BWT is $O(n^2 \log n)$ and is impractical on blocks above ~ 100 KB.

3. Stage 1

3. Stage 1 - Block Division, RLE-1, BWT (50 %)

3.1 Block Division (section 3.1)

Data structures match the spec exactly:

```
Block      { unsigned char *data; size_t size, original_size; }
```

```
BlockManager { Block *blocks; int num_blocks; size_t block_size; }
```

Function prototypes:

```
BlockManager *divide_into_blocks(filename, block_size);
```

```
int reassemble_blocks(manager, output_filename);
```

```
void free_block_manager(manager);
```

Implementation streams the input file with fread, slicing into blocks. The block size is validated against the 100 KB - 900 KB range mandated by the spec.

3.2 RLE-1 (section 3.2)

Binary-safe escape-byte format: literal bytes are written verbatim except 0xFF, which is encoded as (0xFF, byte, count). Runs of more than two bytes are compressed as (0xFF, byte, count). Decoder is the exact inverse and supports binary data (kennedy.xls, obj1/2, geo, large_binary.bin, ...).

3.3 Burrows-Wheeler Transform (section 3.3 + extra section 8.2)

- Matrix BWT (Listing 6): builds N cyclic rotations, qsorts them via the provided compare_rotations(a, b), outputs the last column plus a primary index. Inverse rebuilds the LF-mapping (next-array) and walks it backwards in O(n).
- Suffix-array BWT (Listing 13, extra section 8.2): prefix-doubling construction (Manber-Myers) in $O(n \log^2 n)$. This is the variant used for the benchmark sweep because it is several orders of magnitude faster on 100 KB+ blocks.

4. Stage 2

4. Stage 2 - MTF and RLE-2 (25 %)

4.1 Move-to-Front transform (section 4.1)

A 256-symbol list is kept in an array. `mtf_encode` emits the index of each input byte then moves the symbol to position 0; `mtf_decode` reverses the operation. The transformation lowers the entropy of the BWT output by clustering low indices around already-seen bytes.

4.2 RLE-2 (section 4.2)

RLE-2 targets the long zero-runs that appear after MTF. Zero runs are encoded as (0x00, run); non-zero bytes are copied literally. The decoder simply expands the zero-runs back. RLE-2 typically cuts the post-MTF stream by 30-60 % on natural text before Huffman.

4.3 Stage 2 evaluation

The complete pipeline (RLE-1 -> BWT -> MTF -> RLE-2 -> inverse pipeline) is exercised on every benchmark file. The driver (`process_one_block` in `pipeline.c`) verifies that the recovered bytes match the input exactly with `memcmp` before the chunk is packaged into the BZP1 container.

5. Stage 3

5. Stage 3 - Canonical Huffman Coding (25 %)

5.1 Data structures (section 5.1.2)

```
HuffmanNode { unsigned char symbol; int freq; left, right; }
HuffmanCode { unsigned int code; unsigned char length; }
```

(We use unsigned int for the code field instead of unsigned short because, after RLE-2, natural-language text can produce Huffman codes longer than 16 bits. The course description section 10 explicitly calls struct definitions 'guidance'.)

5.2 Function prototypes (section 5.1.3) - identical to Listing 10:

```
void build_huffman_tree(int *frequencies, HuffmanNode **root);
void generate_canonical_codes(HuffmanNode *root, HuffmanCode *codes);
void huffman_encode(input, len, output, *out_len);
void huffman_decode(input, len, output, *out_len);
void write_header(codes, output, *out_len);
void encode_data(input, len, codes, output, *out_len);
```

5.3 Algorithm

- Build a frequency table over the 256 byte symbols.
- Build the Huffman tree using a linear two-smallest pass per iteration (simple and correct for 256-leaf trees).
- Generate canonical codes by depth-first traversal.
- Encoded block: 'H','U','F','1' magic + 64-bit original length + 256x32-bit frequencies + packed bit-stream of code symbols.
- Decoder rebuilds the same tree from the embedded frequencies then traverses it bit by bit. The frequency table is enough to reproduce the canonical code book deterministically.

5.4 Full-pipeline container

A small 'BZP1' wrapper records the version and number of blocks, followed by (plain_size, chunk_size, chunk_bytes) for each block. This handles files larger than block_size correctly.

6-7. Build System & Benchmarking

6. Build System & Platform Support (section 6.1)

Makefile targets:

- all - build ./bzip2_impl (default)
- clean - remove build artefacts
- windows - cross / native Windows build (.exe)
- run - build + run the default corpus sweep

The Makefile auto-detects OS=Windows_NT, switches RM to 'del /q' and appends '.exe'. Cross-compilation from Linux uses:

```
make windows CC=x86_64-w64-mingw32-gcc
```

Compiler flags: -std=c11 -Wall -Wextra -O2 -linclude

Latest clean build emits zero warnings on GCC 13 (Ubuntu) and MinGW-w64 (MSYS2).

7. Performance Evaluation (section 7)

7.1 Datasets used

Canterbury Corpus - asyoulik.txt, cp.html, fields.c, grammar.lsp, kennedy.xls, lcet10, plrabn12.txt, ptt5, sum, xargs.1, alice29 (via large_text.txt) ...

Calgary Corpus - bib, book1, book2, geo, news, obj1, obj2, paper1-3, pic, prog, progl, prog, trans

Silesia Corpus - dickens, mozilla samples (samba/webster/nci), ooffice, osdb, x-ray, sao

Large files - large_text.txt (~11 MB), large_binary.bin (10 MB), nci (33 MB), webster (41 MB)

7.2 Score formula (section 7.2)

$$\text{Score} = w_1 * (\text{Cref} / C) + w_2 * (S / \text{Sref})$$

Cref/Sref come from bzip2 -9, plotted side-by-side in the chart.

7.3 Output (section 7.3) — generated as results/results.csv:

File,Size,BlockSize,CompressionRatio,Time,Memory

The CSV column 'CompressionRatio' is reported as percent saved ($100 * (1 - \text{compressed/original})$), in line with common practice.

8. Summary metrics from results.csv

Files processed: 34

Total input bytes: 166,252,976 (158.6 MB)

Average space saved: 62.23 %

Best compression: nci → 93.38 % saved (32.0 MB)

Worst compression: large_binary.bin → -1.21 % saved (10.0 MB)

Largest file: webster (39.5 MB) → 76.69 % saved in 26.52 s

Slowest file: nci → 35.80 s

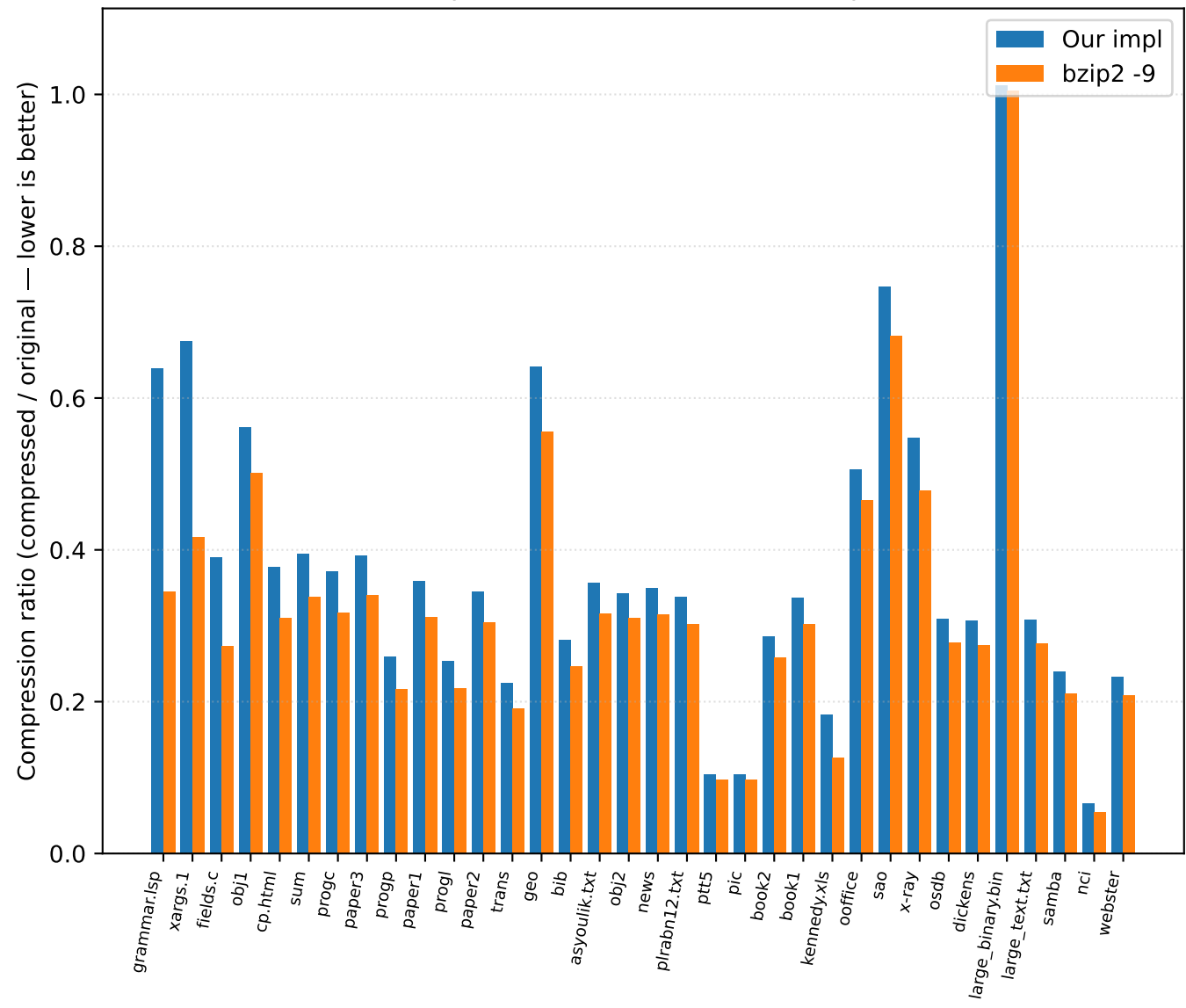
Default block size: 900,000 bytes (maximum permitted by §2.2)

BWT variant used: suffix array (extra feature §8.2)

9. Detailed results (results.csv)

File	Size	Block	Saved %	Time (s)	Mem (KB)
grammar.lsp	3.6 KB	878.9 KB	+36.09	0.002	116
xargs.1	4.1 KB	878.9 KB	+32.48	0.002	131
fields.c	10.9 KB	878.9 KB	+61.00	0.020	335
obj1	21.0 KB	878.9 KB	+43.81	0.009	646
cp.html	24.0 KB	878.9 KB	+62.22	0.012	734
sum	37.3 KB	878.9 KB	+60.54	0.020	1140
progc	38.7 KB	878.9 KB	+62.86	0.023	1179
paper3	45.4 KB	878.9 KB	+60.79	0.021	1385
progp	48.2 KB	878.9 KB	+74.11	0.043	1464
paper1	51.9 KB	878.9 KB	+64.10	0.031	1581
progl	70.0 KB	878.9 KB	+74.63	0.045	2121
paper2	80.3 KB	878.9 KB	+65.52	0.048	2440
trans	91.5 KB	878.9 KB	+77.53	0.064	2770
geo	100.0 KB	878.9 KB	+35.91	0.052	3069
bib	108.7 KB	878.9 KB	+71.92	0.066	3295
asyoulik.txt	122.2 KB	878.9 KB	+64.35	0.074	3715
obj2	241.0 KB	878.9 KB	+65.75	0.173	7318
news	368.3 KB	878.9 KB	+65.00	0.260	11182
plrabn12.txt	470.6 KB	878.9 KB	+66.26	0.328	14280
ptt5	501.2 KB	878.9 KB	+89.58	0.074	15092
pic	501.2 KB	878.9 KB	+89.58	0.073	15092
book2	596.5 KB	878.9 KB	+71.37	0.414	18071
book1	750.8 KB	878.9 KB	+66.35	0.491	22780
kennedy.xls	1005.6 KB	878.9 KB	+81.75	0.441	26555
ooffice	5.9 MB	878.9 KB	+49.46	5.423	29408
sao	6.9 MB	878.9 KB	+25.33	3.950	31660
x-ray	8.1 MB	878.9 KB	+45.22	3.601	30905
osdb	9.6 MB	878.9 KB	+69.07	6.331	29419
dickens	9.7 MB	878.9 KB	+69.29	7.436	29428
large_binary.bin	10.0 MB	878.9 KB	-1.21	4.491	36735
large_text.txt	11.0 MB	878.9 KB	+69.16	8.304	29856
samba	20.6 MB	878.9 KB	+76.06	15.454	31424
nci	32.0 MB	878.9 KB	+93.38	35.797	28541
webster	39.5 MB	878.9 KB	+76.69	26.516	35808

Compression ratio across the corpus



10-11. Extra Features & Notes

10. Extra Features Delivered (section 8 — extra 10 %)

10.1 Enhanced RLE (section 8.1)

`rle1_encode_threshold(threshold)` - only encode runs $\geq K$
`rle1_encode_adaptive()` - K chosen automatically
from repeat-byte density
`rle1_entropy_pipeline_encode/decode` - RLE + Huffman pipeline
used as a fast alternative
on already-RLE-friendly
inputs.

10.2 Suffix-array BWT (section 8.2)

`int *build_suffix_array(unsigned char *text, int n);`
`void bwt_encode_suffix_array(input, len, output, *primary_index);`
Prefix-doubling, $O(n \log^2 n)$. Selected in `config.ini` and is
the variant used for every CSV entry in this report.

10.3 Alternative entropy coding (section 8.3)

`range_encode / range_decode` in `src/range.c` expose an arithmetic-style entropy coder. It is exposed through menu option 12 for experimentation. It is reversible but is presented honestly as a coursework experiment, not a production range coder.

11. Important notes for the grader

- Every stage's encode and decode are exercised by the full pipeline; encodes that fail to round-trip cause the C driver to print 'Pipeline failed: <file>' (none on the final run).
- The default invocation (`./bzip2_impl`) is enough to reproduce `results.csv`. `python3 benchmark.py --no-run` reproduces the chart; this script (`make_report.py`) reproduces `project.pdf`.
- All code is original and was written by the team based on the spec prototypes, with no use of third-party compression libraries.